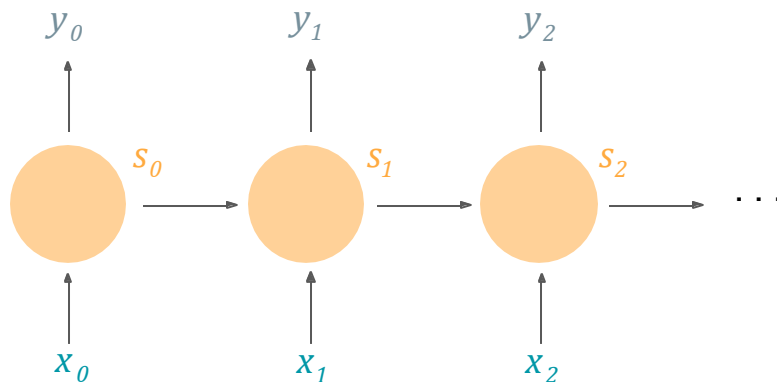


Sequence Modeling with Neural Networks

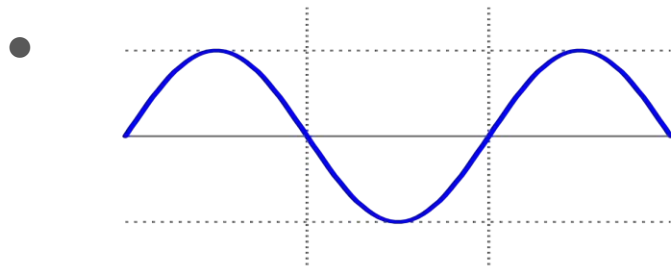
K.Kadri (inspired from Lecture MIT 6.S191)



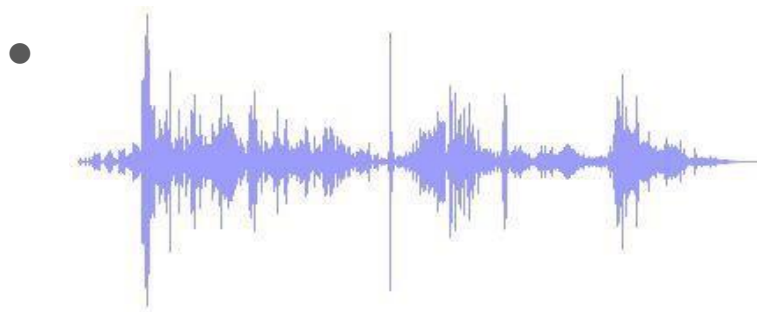
What is a sequence?

- “I took the dog for a walk this morning.”

sentence



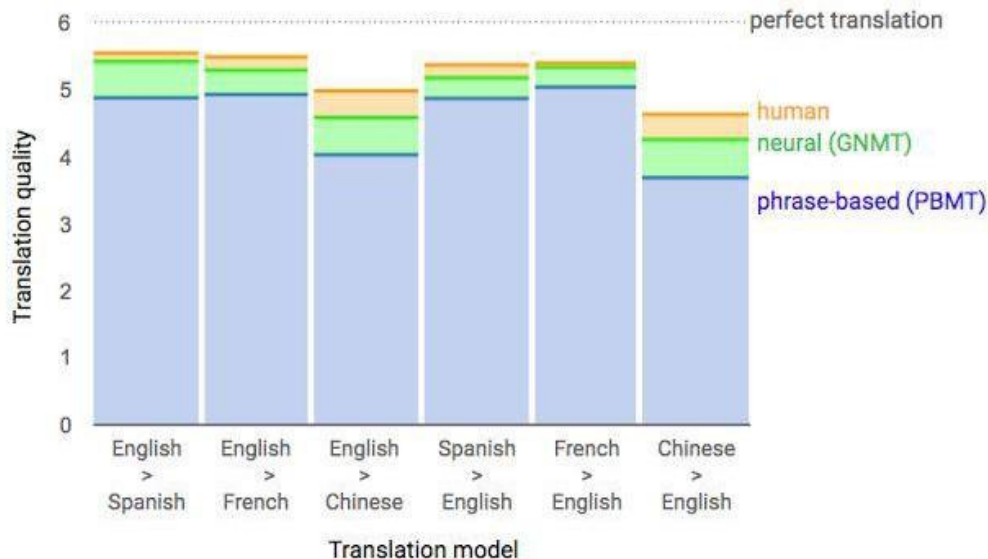
function



speech waveform

Successes of deep models

Machine translation



Question Answering

Super Bowl 50 was an American football game to determine the champion of the National Football League (NFL) for the 2015 season. The American Football Conference (AFC) champion Denver Broncos defeated the National Football Conference (NFC) champion Carolina Panthers 24–10 to earn their third Super Bowl title. The game was played on February 7, 2016, at Levi's Stadium in the San Francisco Bay Area at Santa Clara, California. As this was the 50th Super Bowl, the league emphasized the "golden anniversary" with various gold-themed initiatives, as well as temporarily suspending the tradition of naming each Super Bowl game with Roman numerals (under which the game would have been known as "Super Bowl L"), so that the logo could prominently feature the Arabic numerals 50.

Super Bowl 50 decided the NFL champion for what season?

Ground Truth Answers: 2015 the 2015 season 2015

Prediction: 2015

Left:

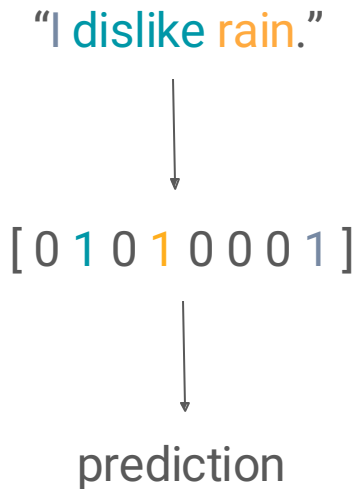
<https://research.googleblog.com/2016/09/a-neural-network-for-machine.html>

Right:

<https://rajpurkar.github.io/SQuAD-explorer/>

how do we model sequences?

idea: represent a sequence as a *bag of words*



problem: bag of words does not preserve order

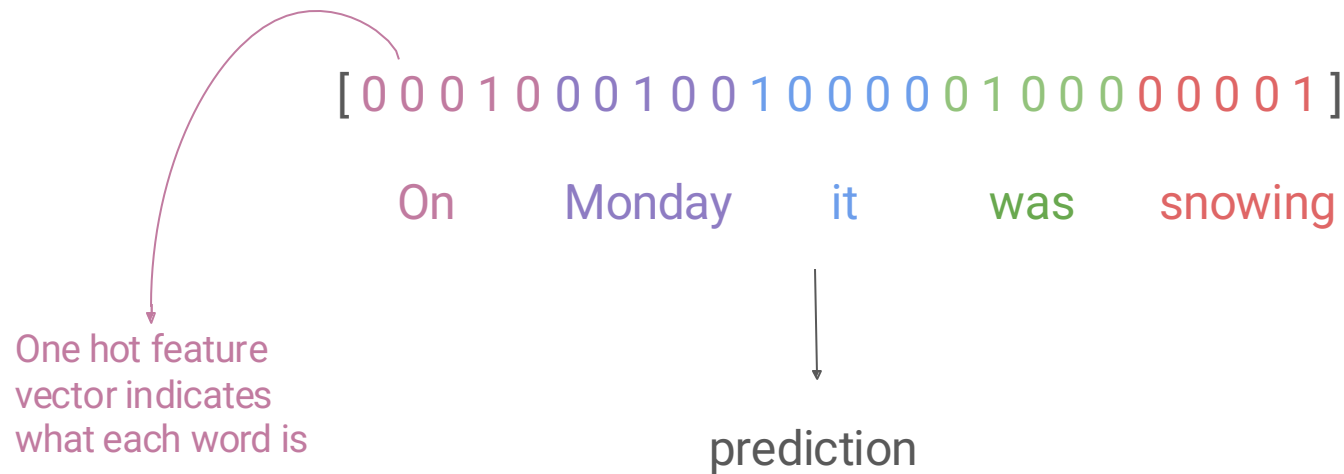
problem: bag of words does not preserve order

“The food was good, not bad at all.”

VS

“The food was bad, not good at all.”

idea: maintain an ordering within feature vector



problem: hard to deal with different word orders

“On Monday, it was snowing.”

VS

“It was snowing on Monday.”

problem: hard to deal with different word orders

[0 0 0 1 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1]

On Monday it was snowing

vs

[1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0]

It was snowing on Monday

problem: hard to deal with different word orders

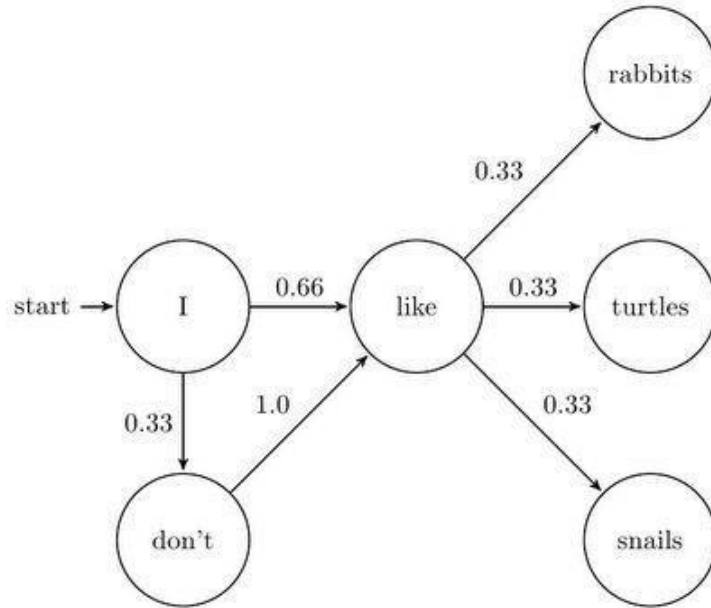
“On Monday it was snowing.”

VS

“It was snowing on Monday.”

We would have to **relearn the rules of language** at each point in the sentence.

idea: markov models



problem: we can't model long-term dependencies

markov assumption: each state depends only on the last state.

problem: we can't model long-term dependencies

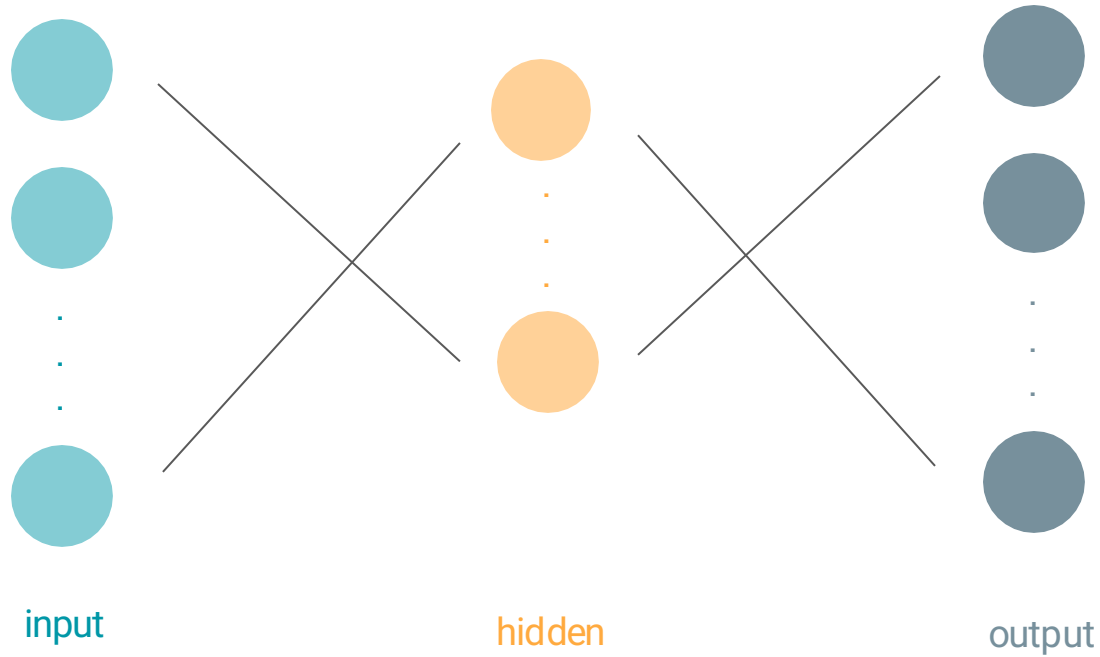
“In France, I had a great time and I learnt some of the _____
language.”

We need information from the far past and future to accurately guess the correct word.

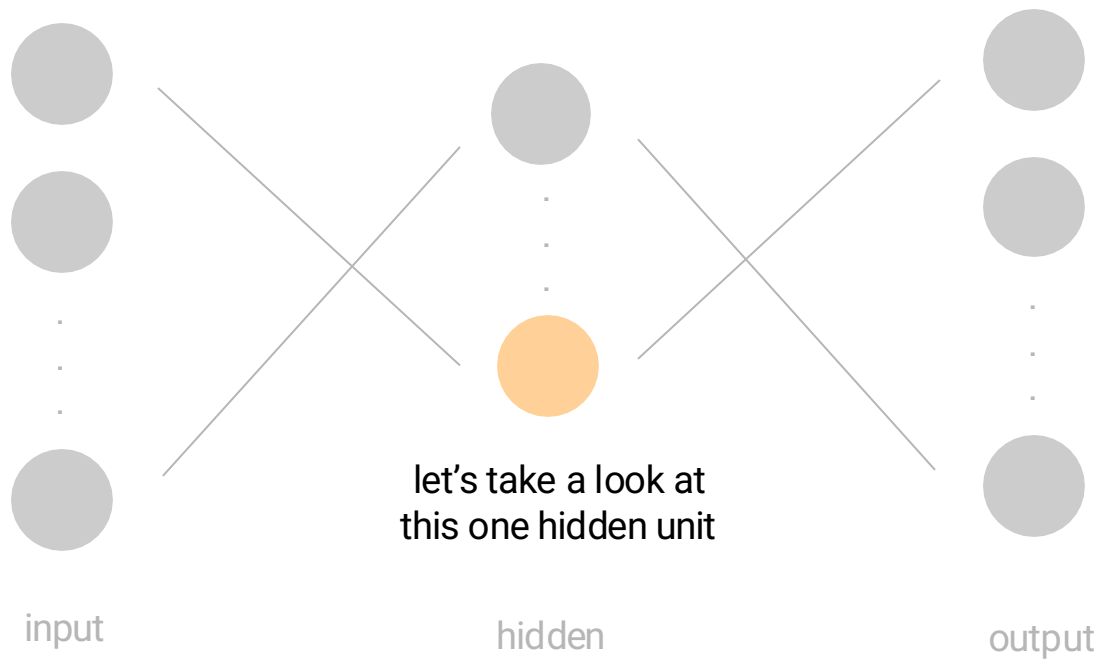
let's turn to **recurrent neural networks!** (RNNs)

1. to maintain word order
2. to share parameters across the sequence
3. to keep track of long-term dependencies

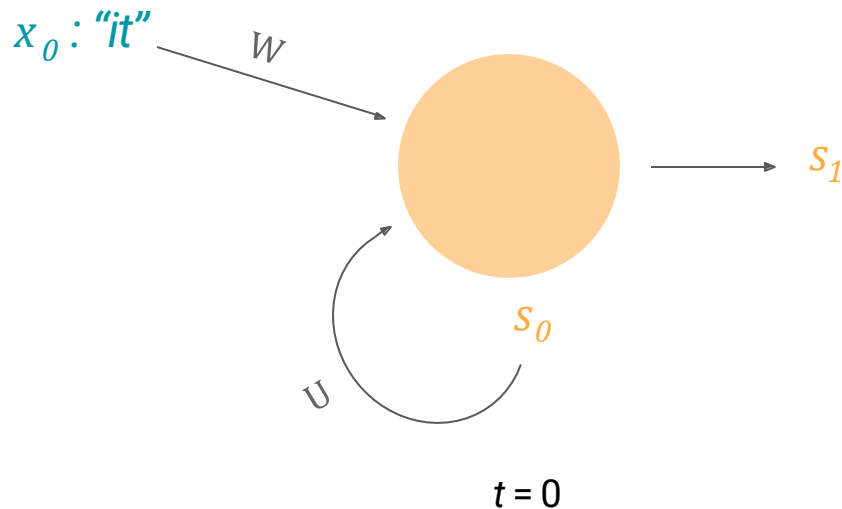
example network:



example network:



RNNS **remember** their previous state:

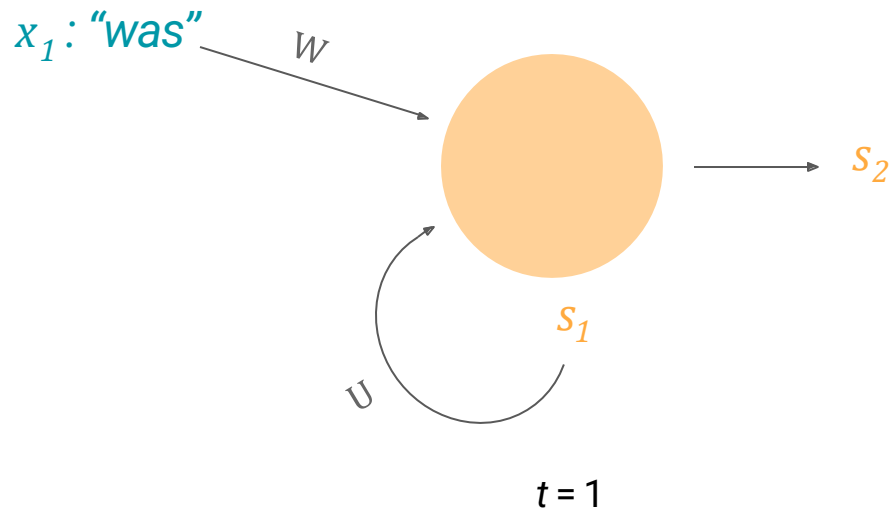


x_0 : vector representing first word
 s_0 : cell state at $t = 0$ (some initialization)
 s_1 : cell state at $t = 1$

$$s_1 = \tanh(Wx_0 + Us_0)$$

W , U : weight matrices

RNNS **remember** their previous state:



x_1 : vector representing second word

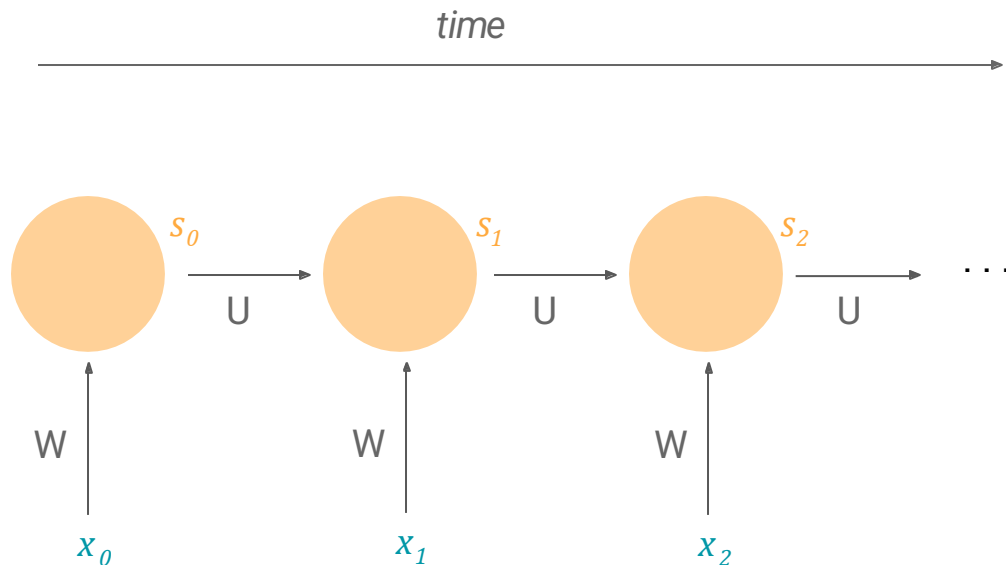
s_1 : cell state at $t = 1$

s_2 : cell state at $t = 2$

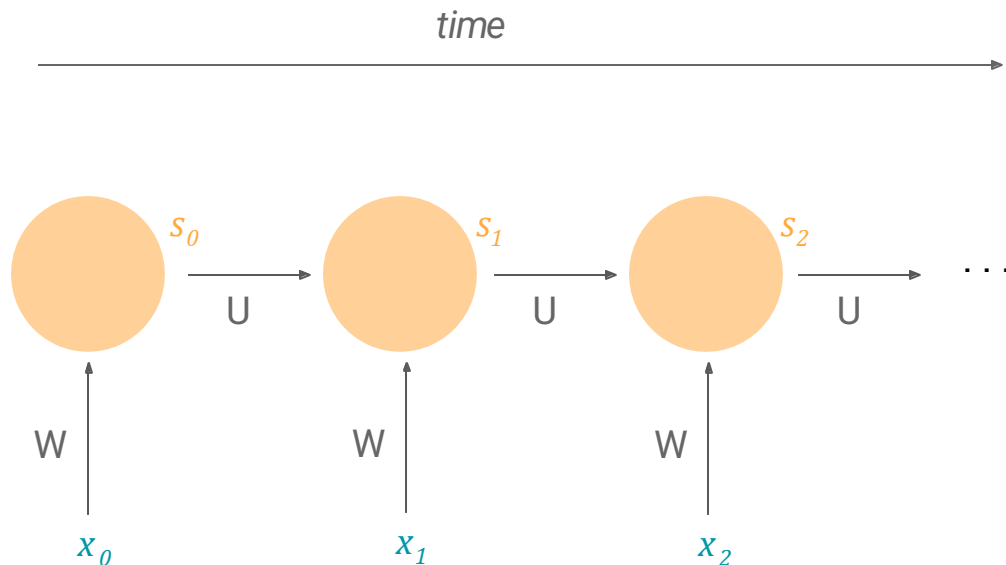
$$s_2 = \tanh(Wx_1 + Us_1)$$

W , U : weight matrices

“unfolding” the RNN across time:

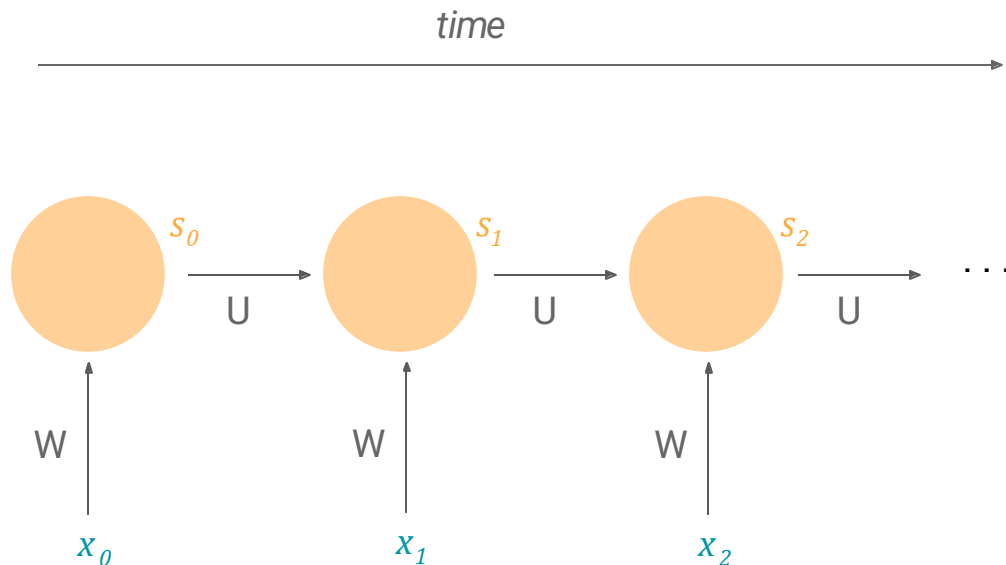


“unfolding” the RNN across time:



notice that W and U stay the same!

“unfolding” the RNN across time:



s_n can contain
information from all
past timesteps

possible task: language model

all the works of
shakespeare



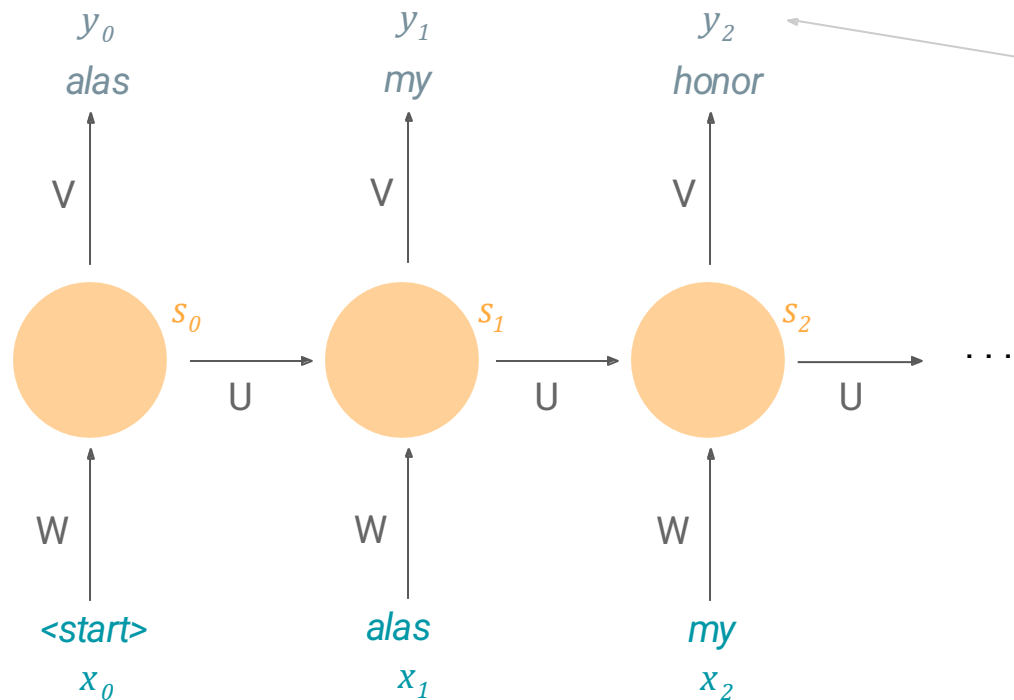
*language
model*



KING LEAR:

O, if you were a feeble sight, the
courtesy of your law,
Your sight and several breath, will
wear the gods
With his heads, and my hands are
wonder'd at the deeds,
So drop upon your lordship's head,
and your opinion
Shall be against your honour.

possible task: language model



y_i is actually a probability distribution over possible next words, aka a *softmax*

possible task: language model

King James Bible,
Structure and Interpretation
of Computer Programs



*language
model*

37:29 The righteous shall inherit
the land, and leave it for an
inheritance unto the children of
Gad according to the number of
steps that is linear in b .

hath it not been for the singular
taste of old Unix, “new Unix”
would not exist.

<http://kingjamesprogramming.tumblr.com/>

possible task: classification (i.e. sentiment)



@HVSVN



Don't fly with @British_Airways.
They can't keep track of your
luggage.



:(



Kim Kardashian ✓
@KimKardashian



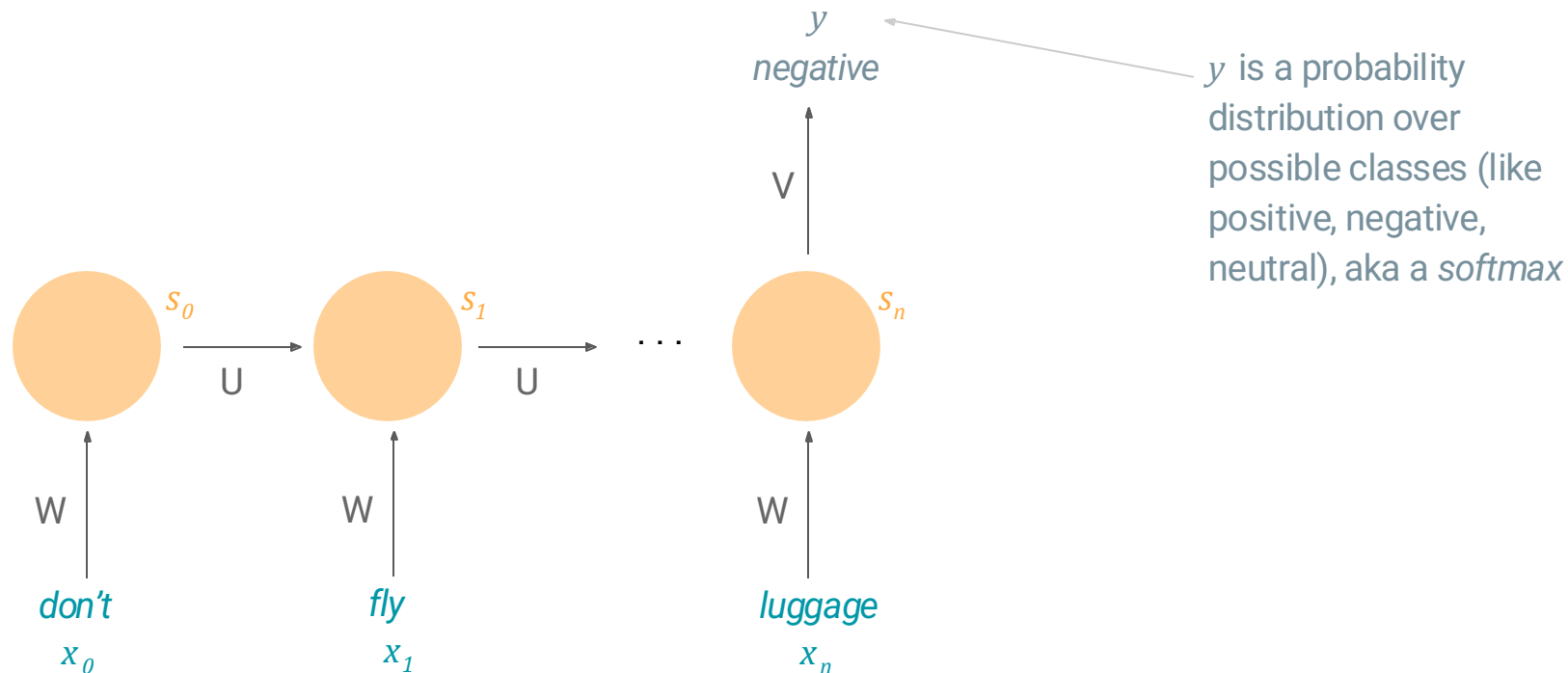
Following

Happy Birthday to my best friend, the ♥ of
my life, my soul!!!! I love you beyond words!
[instagram.com/p/aTgfl-OS-a/](https://www.instagram.com/p/aTgfl-OS-a/)

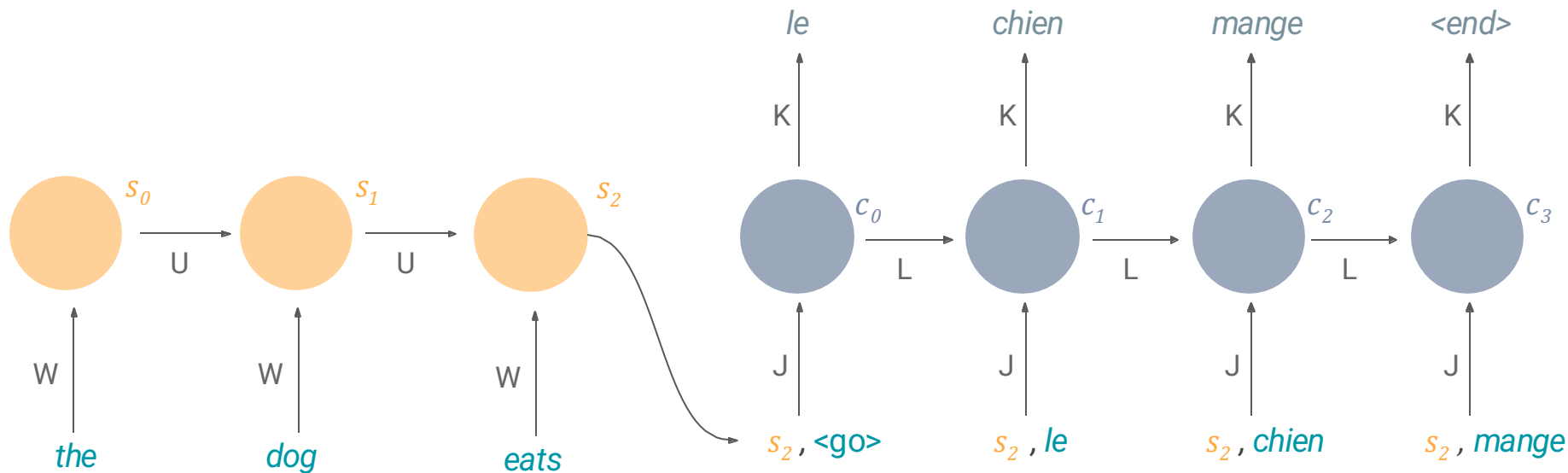


:)

possible task: classification (i.e. sentiment)



possible task: machine translation



how do we **train** an RNN?

how do we **train** an RNN?

backpropagation!

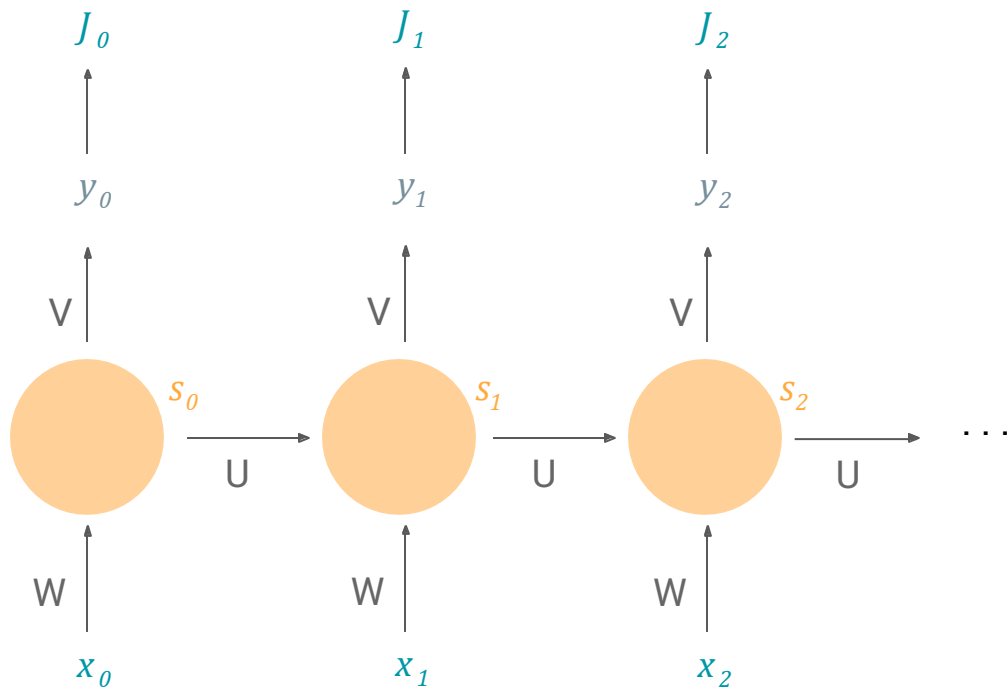
(through time)

remember: **backpropagation**

1. **take the derivative** (gradient) of the loss with respect to each parameter
2. **shift parameters in the opposite direction** in order to minimize loss

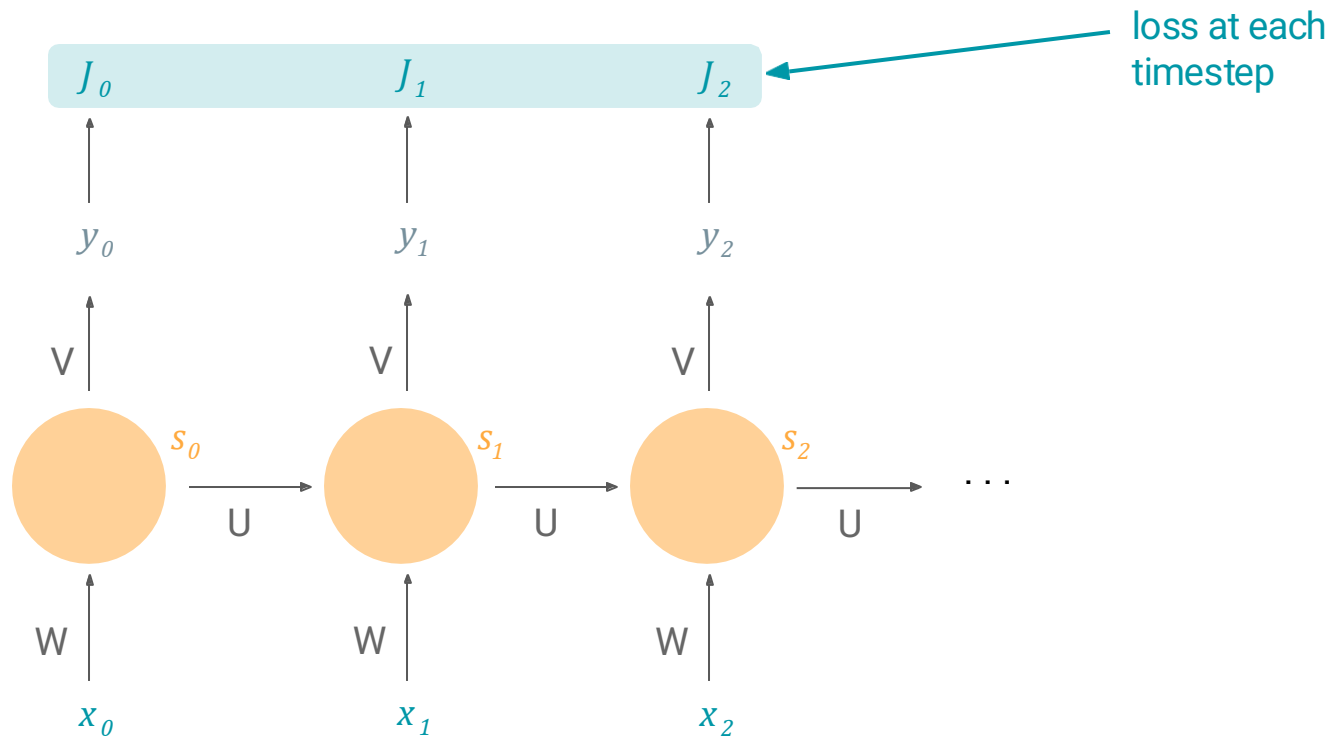
we have a **loss at each timestep**:

(since we're making a prediction at each timestep)



we have a **loss at each timestep**:

(since we're making a prediction at each timestep)



we **sum the losses** across time:

$$\text{loss at time } t = J_t(\Theta)$$



Θ = our
parameters, like
weights

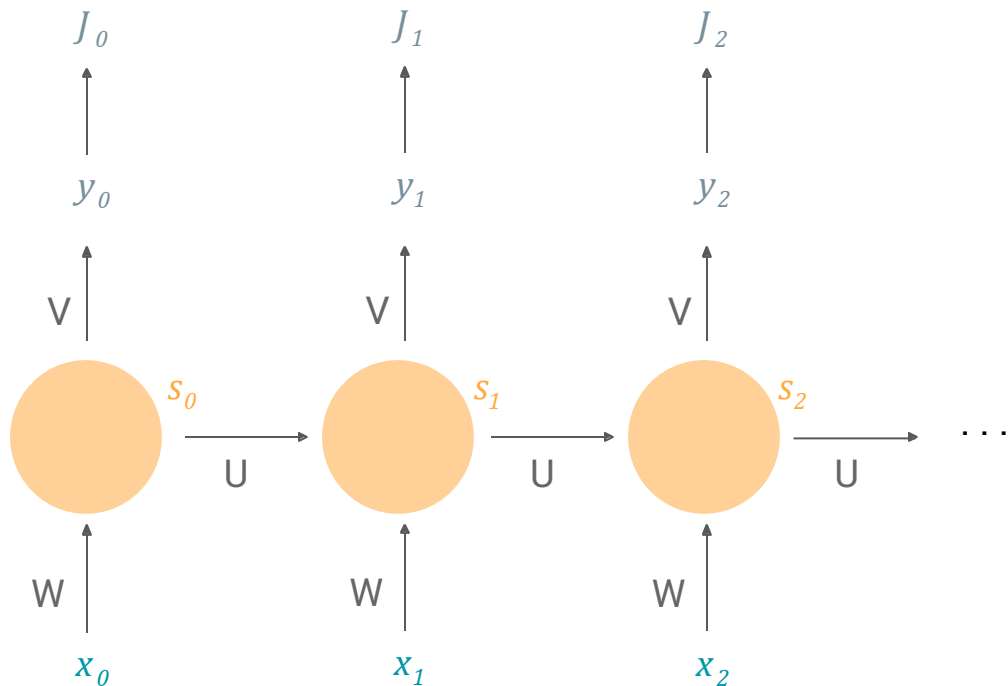
$$\text{total loss} = J(\Theta) = \sum_t J_t(\Theta)$$

what are our **gradients**?

we sum gradients across time for each
parameter P :

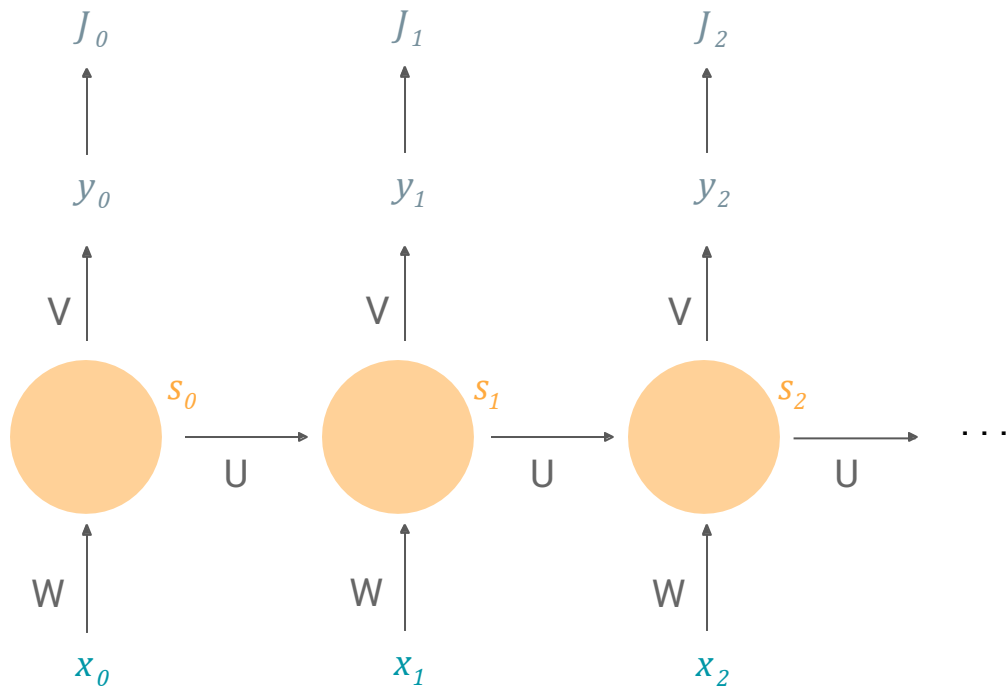
$$\frac{\partial J}{\partial P} = \sum_t \frac{\partial J_t}{\partial P}$$

let's try it out for W with the **chain rule**:



$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

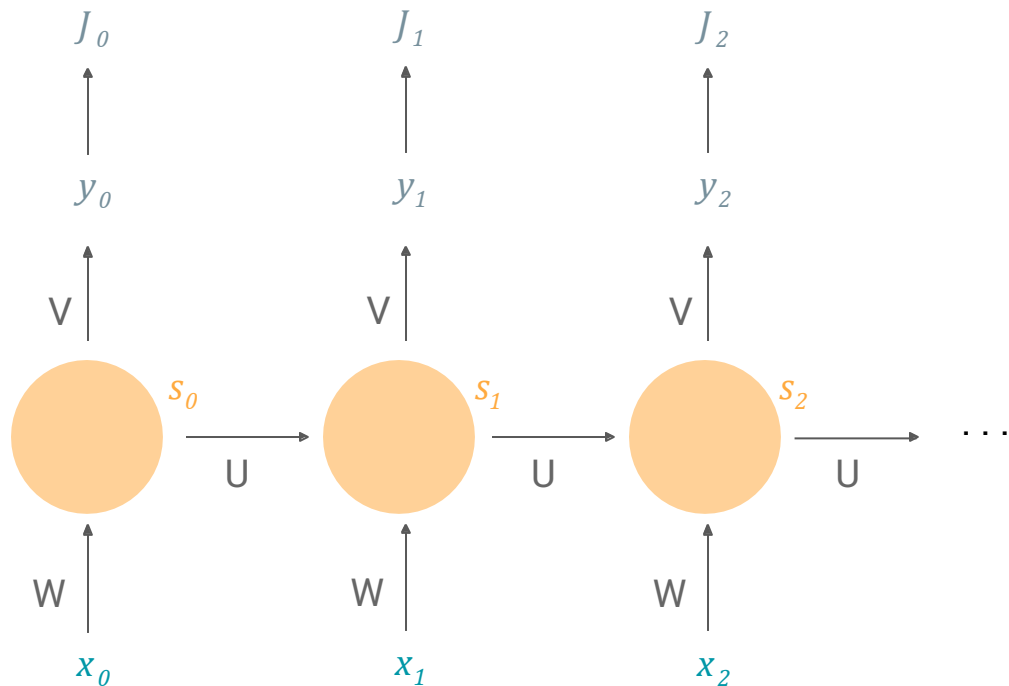
let's try it out for W with the **chain rule**:



$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

so let's take a single timestep t :

let's try it out for W with the **chain rule**:

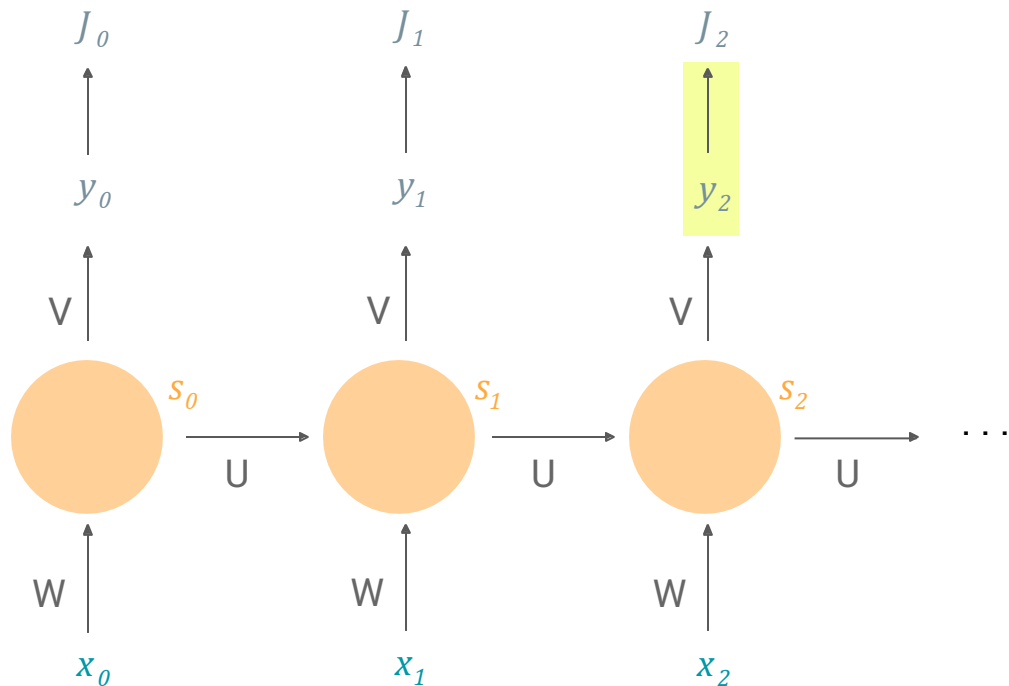


$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

so let's take a single timestep t :

$$\frac{\partial J_2}{\partial W}$$

let's try it out for W with the **chain rule**:

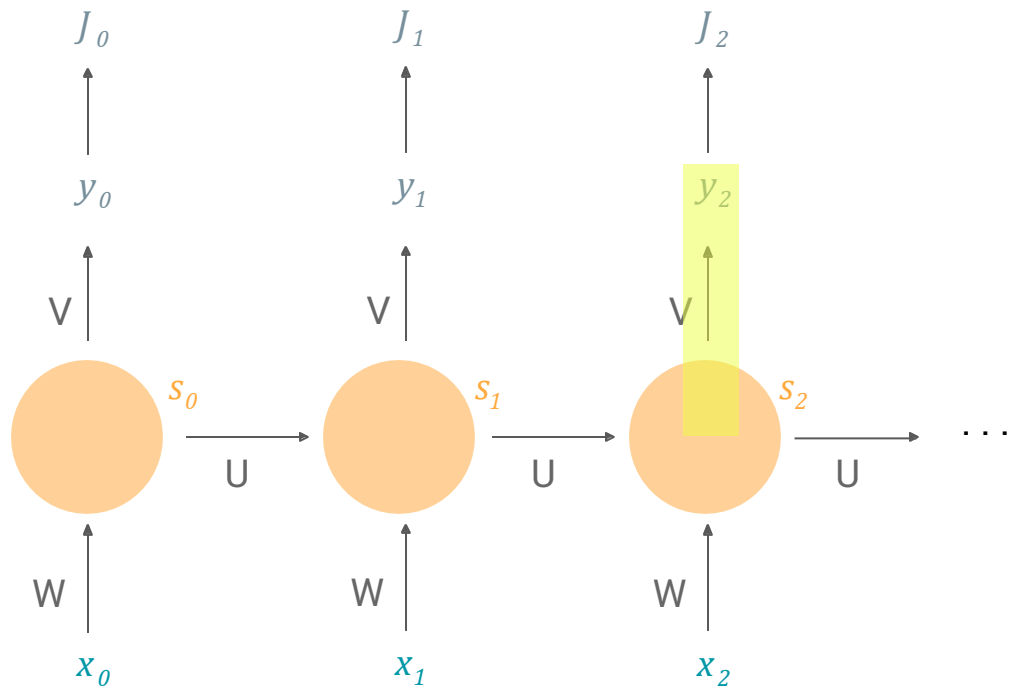


$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

so let's take a single timestep t :

$$\frac{\partial J_2}{\partial W} = \frac{\partial J_2}{\partial y_2}$$

let's try it out for W with the **chain rule**:

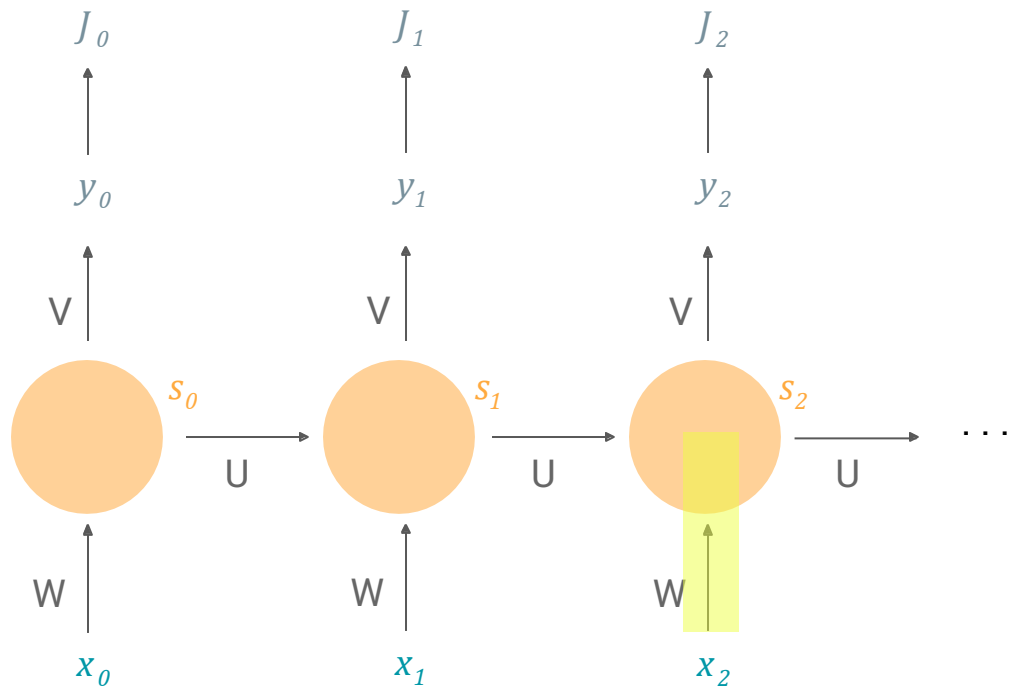


$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

so let's take a single timestep t :

$$\frac{\partial J_2}{\partial W} = \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2}$$

let's try it out for W with the **chain rule**:

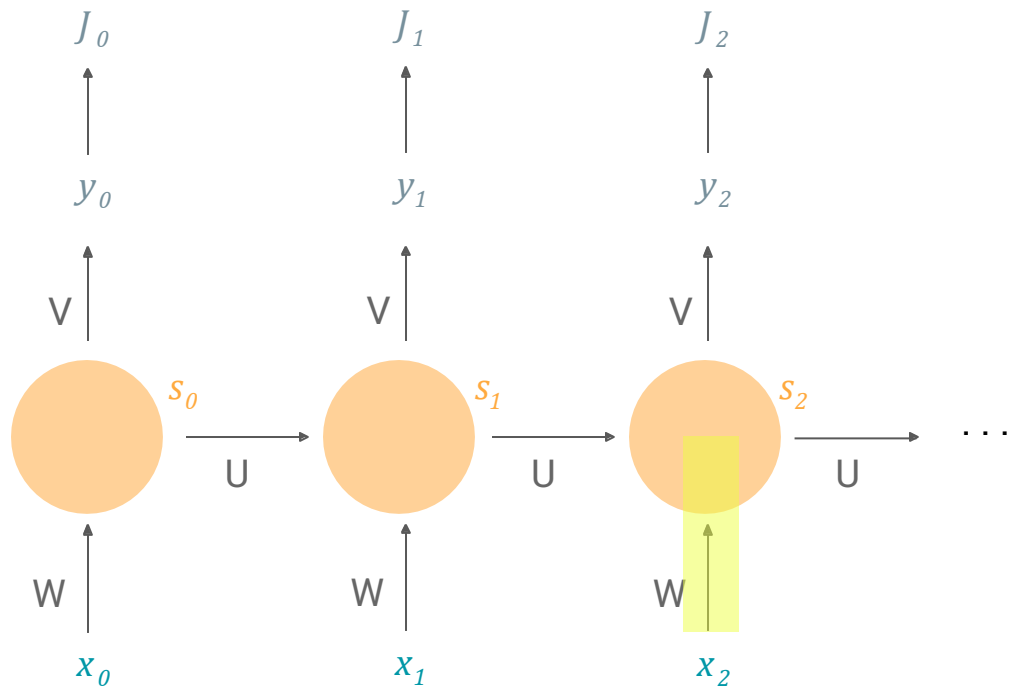


$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

so let's take a single timestep t :

$$\frac{\partial J_2}{\partial W} = \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2} \frac{\partial s_2}{\partial W}$$

let's try it out for W with the **chain rule**:



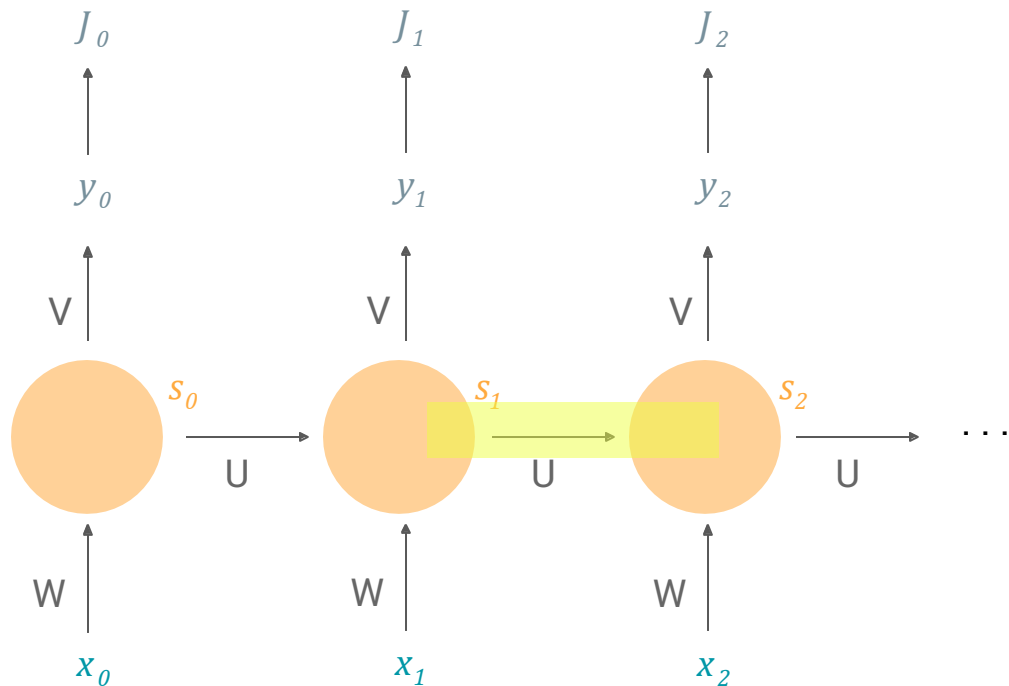
$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

so let's take a single timestep t :

$$\frac{\partial J_2}{\partial W} = \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2} \frac{\partial s_2}{\partial W}$$

but wait...

let's try it out for W with the **chain rule**:



$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

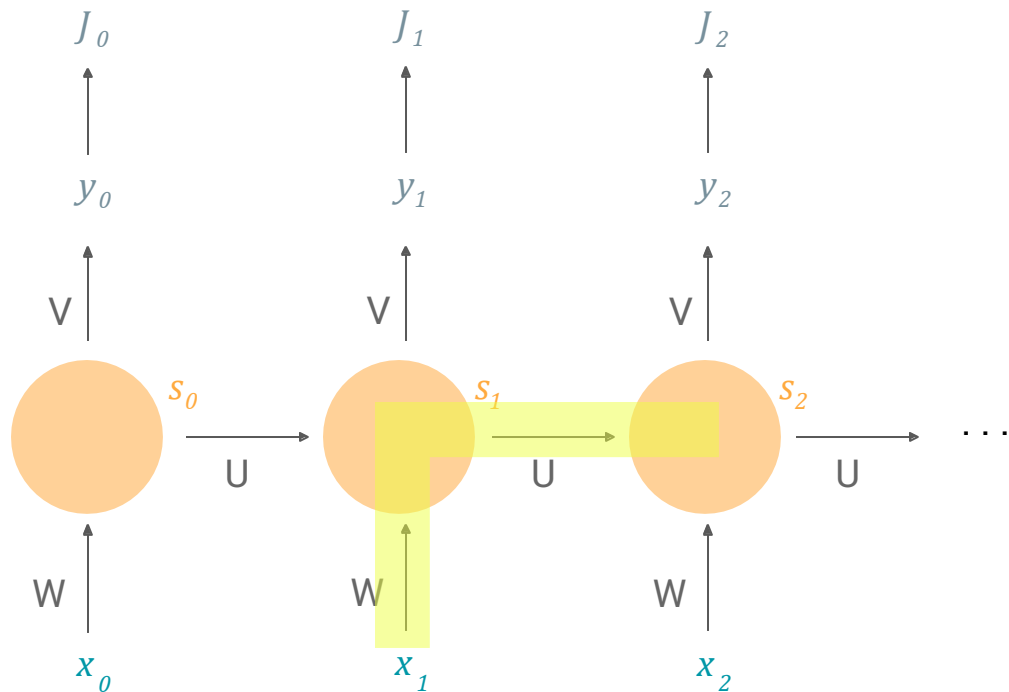
so let's take a single timestep t :

$$\frac{\partial J_2}{\partial W} = \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2} \frac{\partial s_2}{\partial W}$$

but wait...

$$s_2 = \tanh(U s_1 + W x_2)$$

let's try it out for W with the **chain rule**:



$$\frac{\partial J}{\partial W} = \sum_t \frac{\partial J_t}{\partial W}$$

so let's take a single timestep t :

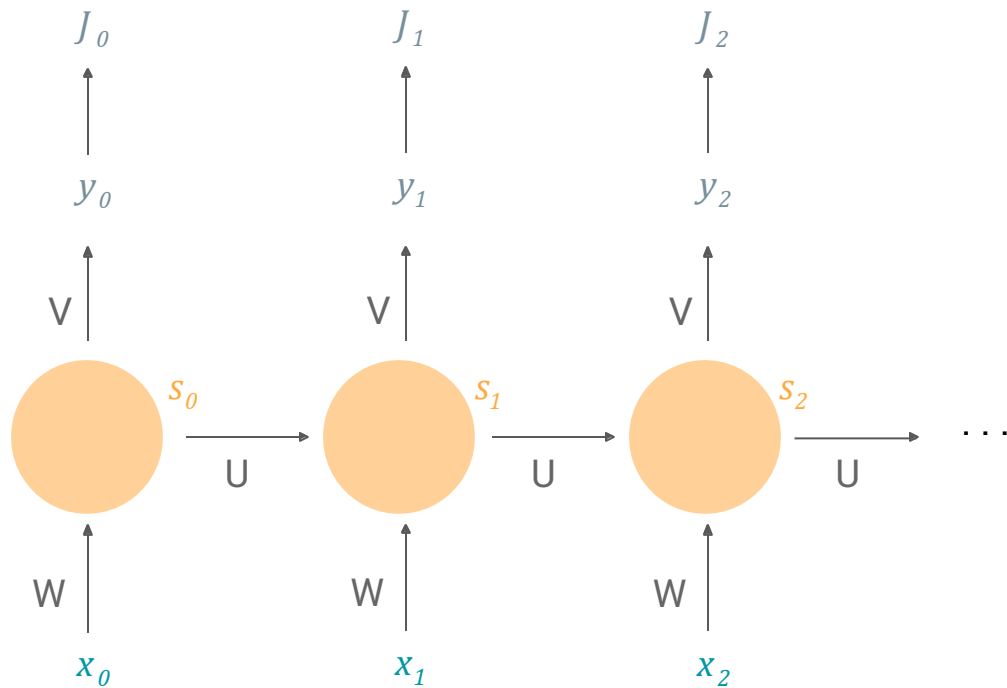
$$\frac{\partial J_2}{\partial W} = \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2} \frac{\partial s_2}{\partial W}$$

but wait...

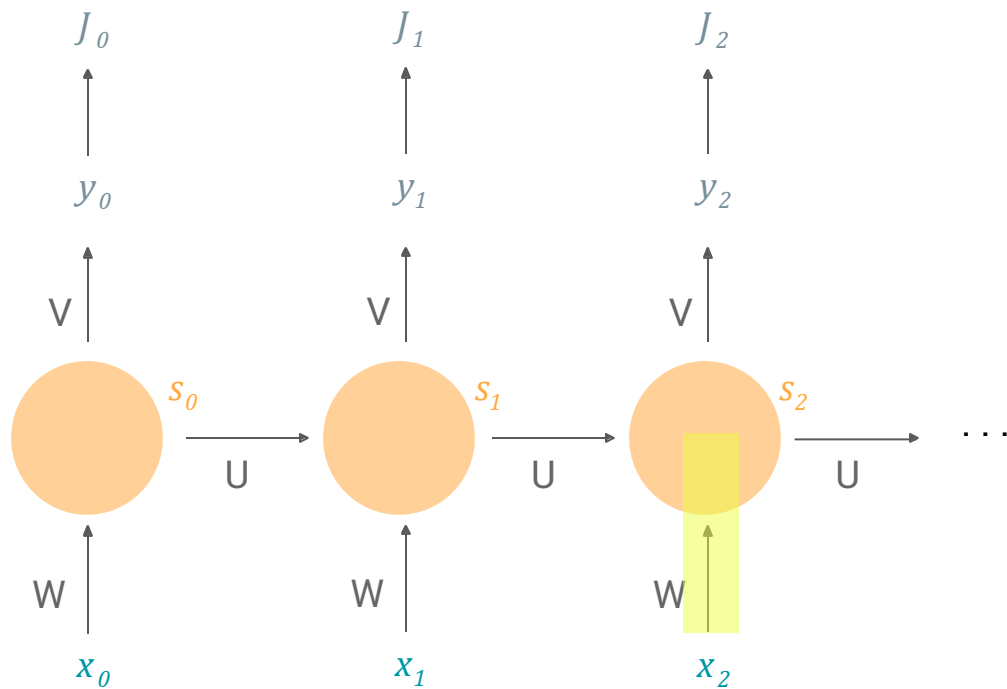
$$s_2 = \tanh(U s_1 + W x_2)$$

s_1 also depends on W so we can't just treat $\frac{\partial s_2}{\partial W}$ as a constant!

how does s_2 depend on W ?

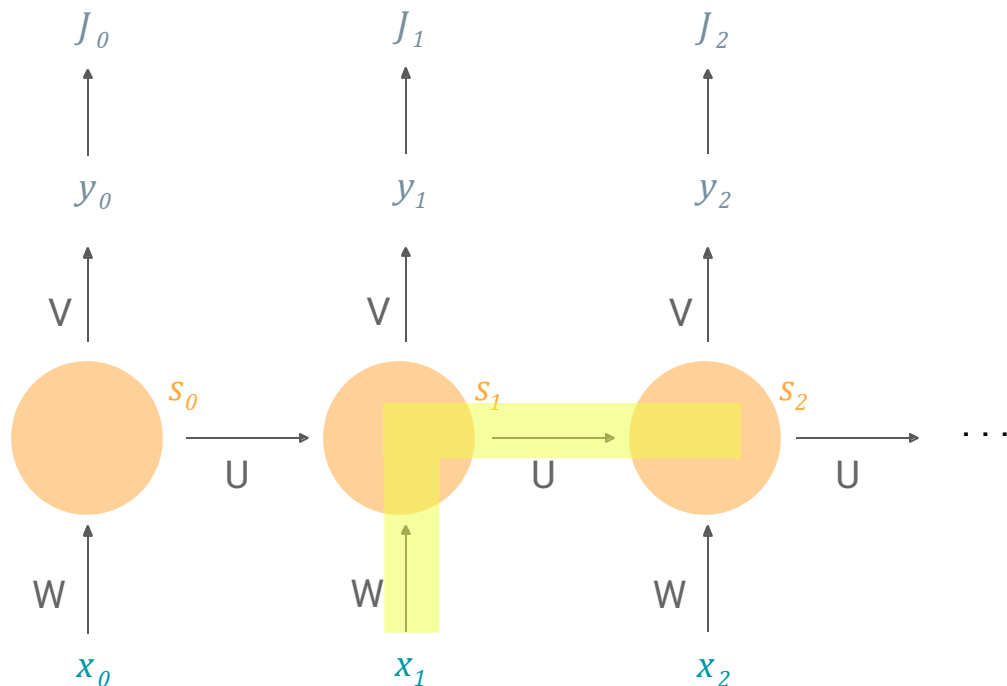


how does s_2 depend on W ?



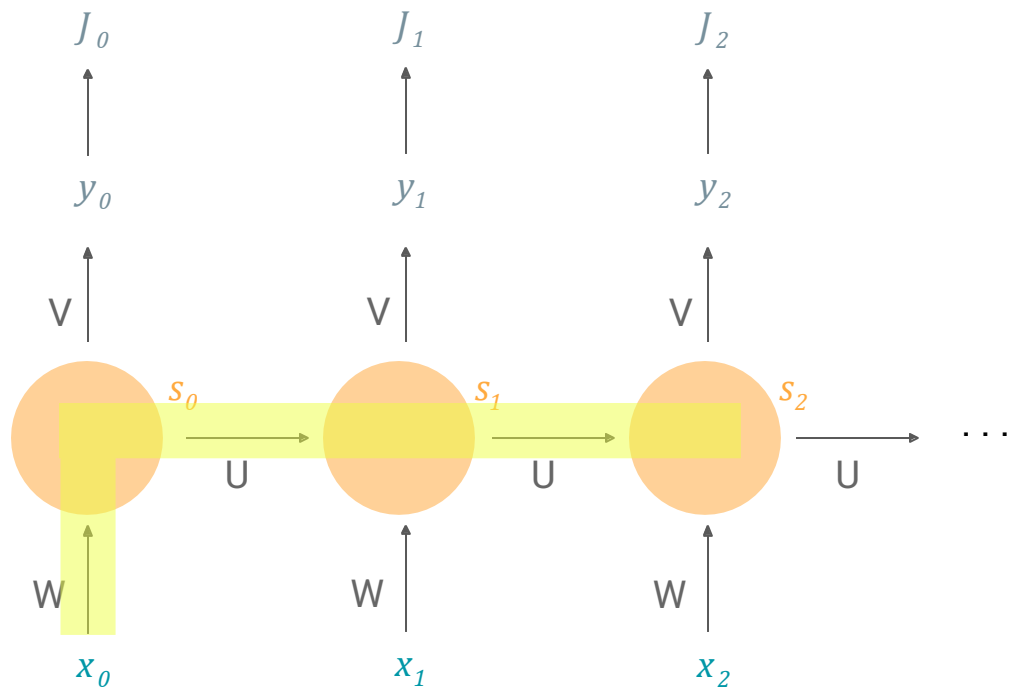
$$\frac{\partial s_2}{\partial W}$$

how does s_2 depend on W ?



$$\frac{\partial s_2}{\partial W} + \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial W}$$

how does s_2 depend on W ?



$$\begin{aligned} & \frac{\partial s_2}{\partial W} \\ & + \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial W} \\ & + \frac{\partial s_2}{\partial s_0} \frac{\partial s_0}{\partial W} \end{aligned}$$

backpropagation through time:

$$\frac{\partial J_2}{\partial W} = \sum_{k=0}^2 \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2} \underbrace{\frac{\partial s_2}{\partial s_k} \frac{\partial s_k}{\partial W}}_{\text{Contributions of } W \text{ in previous timesteps to the error at timestep } t}$$

Contributions of W in previous timesteps to the error at timestep t

backpropagation through time:

$$\frac{\partial J_t}{\partial W} = \sum_{k=0}^t \frac{\partial J_t}{\partial y_t} \frac{\partial y_t}{\partial s_t} \underbrace{\frac{\partial s_t}{\partial s_k} \frac{\partial s_k}{\partial W}}_{\text{Contributions of } W \text{ in previous timesteps to the error at timestep } t}$$

Contributions of W in previous timesteps to the error at timestep t

why are RNNs **hard to train**?

problem: vanishing gradient

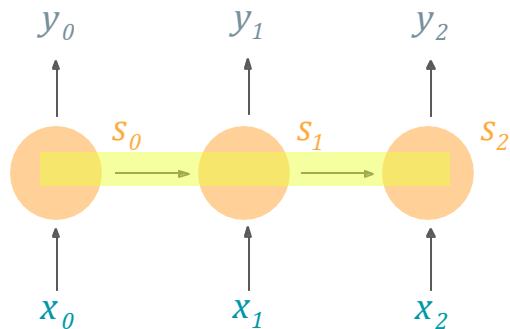
$$\frac{\partial J_2}{\partial W} = \sum_{k=0}^2 \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2} \frac{\partial s_2}{\partial s_k} \frac{\partial s_k}{\partial W}$$

problem: vanishing gradient

$$\frac{\partial J_2}{\partial W} = \sum_{k=0}^2 \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2} \frac{\partial s_2}{\partial s_k} \frac{\partial s_k}{\partial W}$$

problem: vanishing gradient

$$\frac{\partial J_2}{\partial W} = \sum_{k=0}^2 \frac{\partial J_2}{\partial y_2} \frac{\partial y_2}{\partial s_2} \boxed{\frac{\partial s_2}{\partial s_k}} \frac{\partial s_k}{\partial W}$$



at $k = 0$:

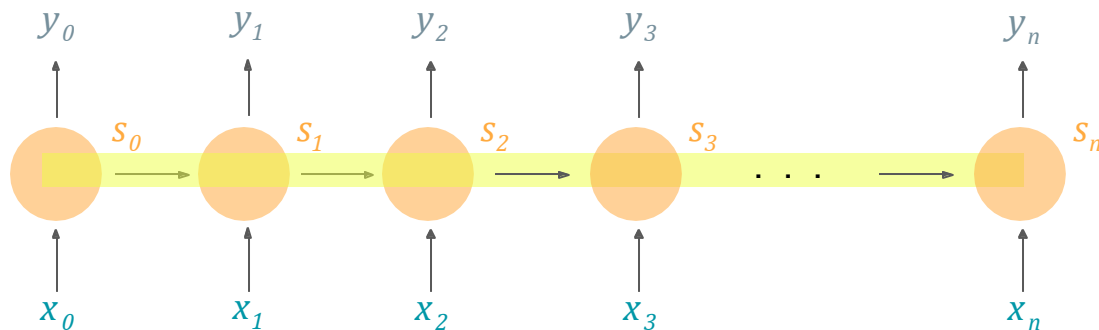
$$\boxed{\frac{\partial s_2}{\partial s_0} = \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial s_0}}$$

problem: vanishing gradient

$$\frac{\partial J_n}{\partial W} = \sum_{k=0}^n \frac{\partial J_n}{\partial y_n} \frac{\partial y_n}{\partial s_n} \frac{\partial s_n}{\partial s_k} \frac{\partial s_k}{\partial W}$$

A blue arrow points from the term $\frac{\partial s_n}{\partial s_k}$ in the equation to a light blue box containing the chain of partial derivatives:

$$\frac{\partial s_n}{\partial s_{n-1}} \frac{\partial s_{n-1}}{\partial s_{n-2}} \cdots \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial s_0}$$



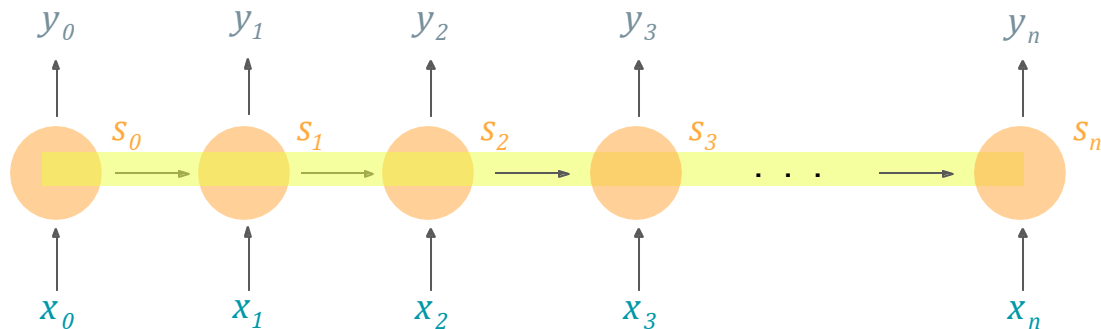
problem: vanishing gradient

$$\frac{\partial J_n}{\partial W} = \sum_{k=0}^n \frac{\partial J_n}{\partial y_n} \frac{\partial y_n}{\partial s_n} \frac{\partial s_n}{\partial s_k} \frac{\partial s_k}{\partial W}$$

The term $\frac{\partial s_n}{\partial s_k} \frac{\partial s_k}{\partial W}$ is highlighted in a light blue box. An arrow points from this box to a larger light blue box containing the expanded product of gradients:

$$\frac{\partial s_n}{\partial s_{n-1}} \frac{\partial s_{n-1}}{\partial s_{n-2}} \cdots \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial s_0}$$

as the gap between timesteps gets bigger, this product gets longer and longer!



problem: vanishing gradient

$$\frac{\partial s_n}{\partial s_{n-1}} \frac{\partial s_{n-1}}{\partial s_{n-2}} \cdot \cdot \cdot \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial s_0}$$

problem: vanishing gradient

what are each of these terms?



$$\frac{\partial s_n}{\partial s_{n-1}} \frac{\partial s_{n-1}}{\partial s_{n-2}} \cdot \cdot \cdot \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial s_0}$$

problem: vanishing gradient

what are each of these terms? →

$$\frac{\partial s_n}{\partial s_{n-1}} = W^T \text{diag}[f'(W_{s_{j-1}} + Ux_j)]$$

W = sampled from
standard normal
distribution = mostly < 1

f = tanh or sigmoid so $f' < 1$

$$\frac{\partial s_n}{\partial s_{n-1}} \frac{\partial s_{n-1}}{\partial s_{n-2}} \cdot \cdot \cdot \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial s_0}$$

problem: vanishing gradient

what are each of these terms? →

$$\frac{\partial s_n}{\partial s_{n-1}} \frac{\partial s_{n-1}}{\partial s_{n-2}} \cdot \cdot \cdot \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial s_0}$$

$$\frac{\partial s_n}{\partial s_{n-1}} = W^T \text{diag}[f'(W_{s_{j-1}} + Ux_j)]$$

W = sampled from
standard normal
distribution = mostly < 1

f = tanh or sigmoid so $f' < 1$

we're multiplying a lot of **small numbers** together.

we're multiplying a lot of **small numbers** together.

so what?

errors due to further back timesteps have increasingly **smaller gradients**.

so what?

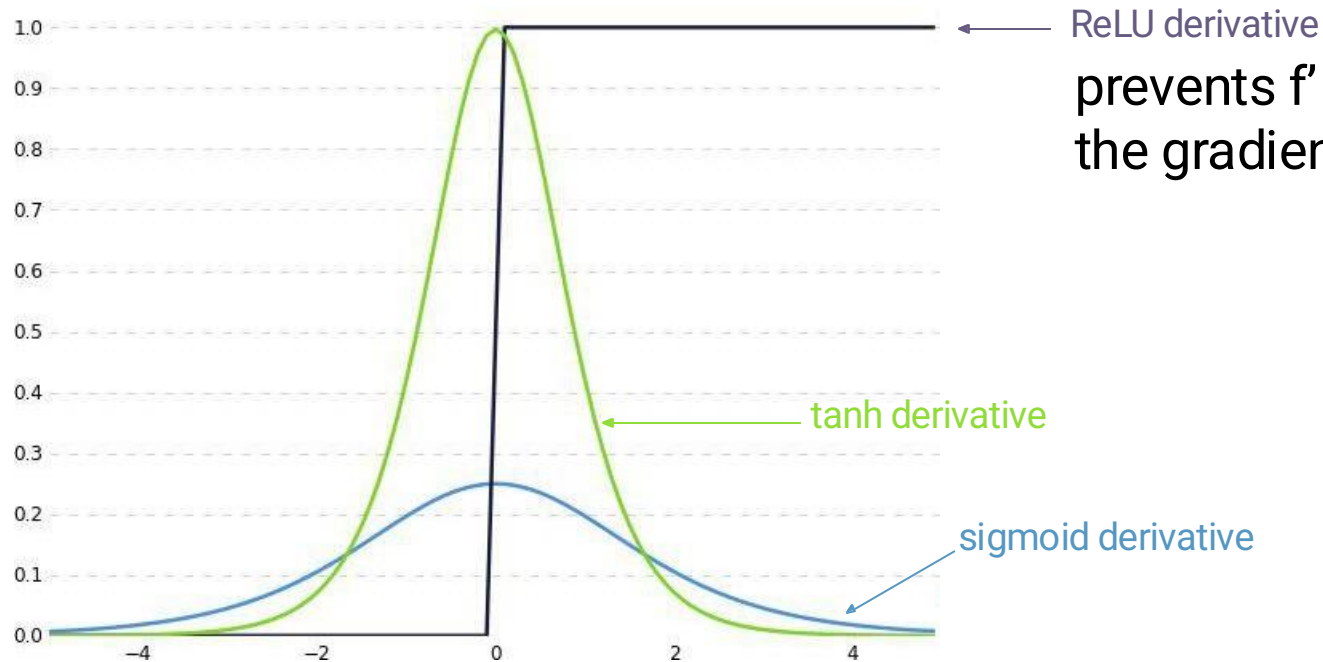
parameters become biased to **capture shorter-term** dependencies.

“In France, I had a great time and I learnt some of the _____language.”



our parameters are not trained to capture long-term dependencies, so the word we predict will mostly depend on the previous few words, not much earlier ones

solution #1: activation functions



ReLU derivative
prevents f' from shrinking
the gradients

tanh derivative

sigmoid derivative

solution #2: initialization

weights initialized to identity matrix

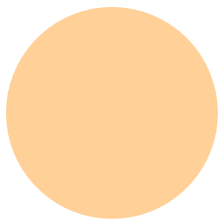
biases initialized to zeros

$$\longrightarrow I_n = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix}$$

prevents W from shrinking the gradients

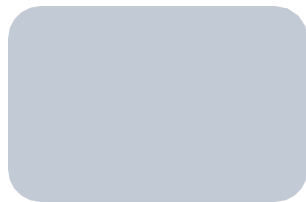
solution #3: **gated cells**

rather each node being just a simple RNN cell, make each node a more **complex unit with gates** controlling what information is passed through.



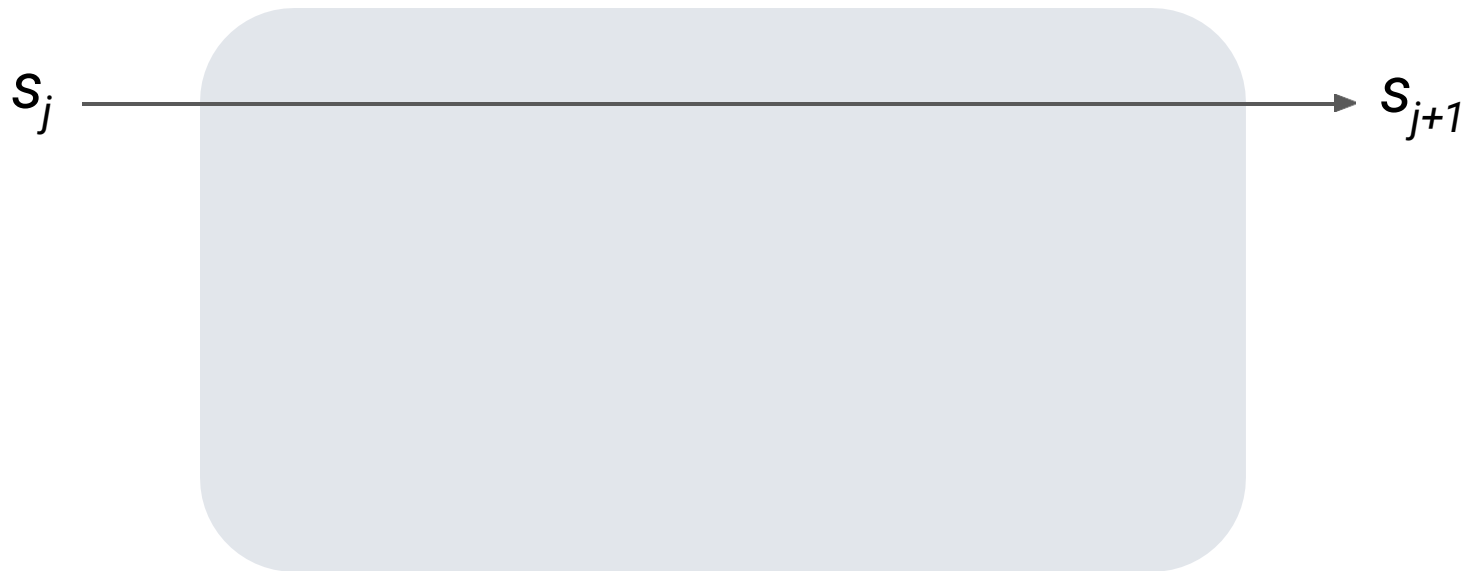
RNN

vs



LSTM, GRU, etc

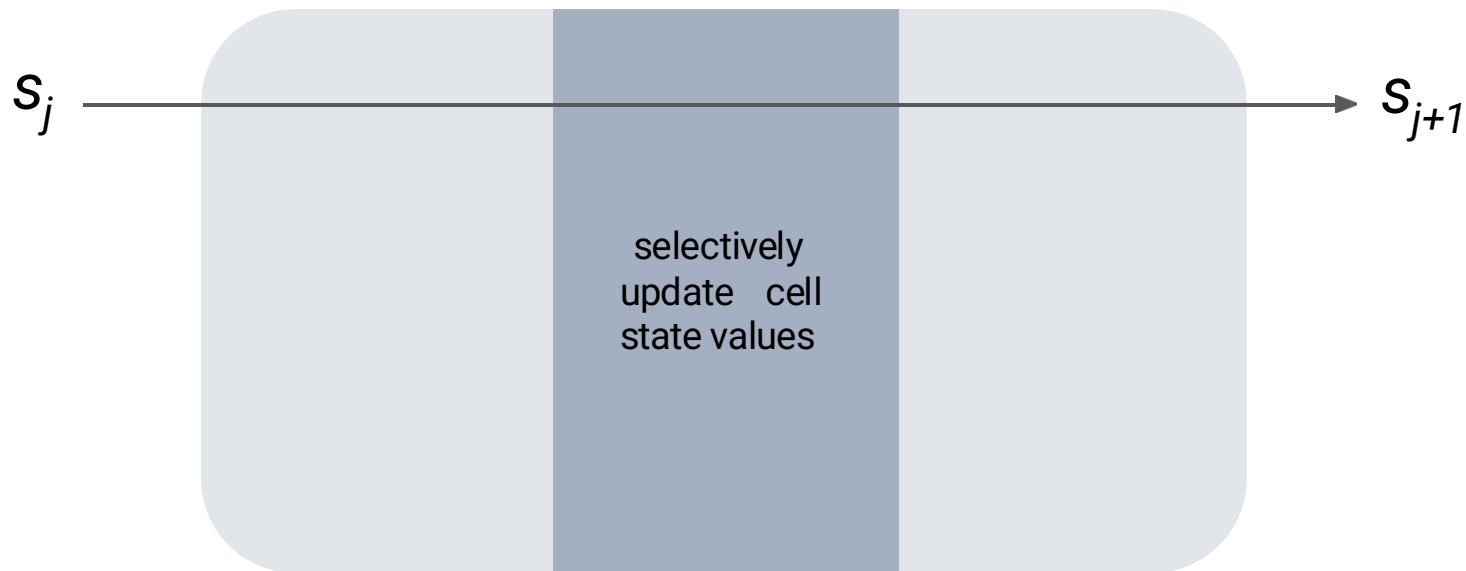
solution #3: more on **LSTMs**



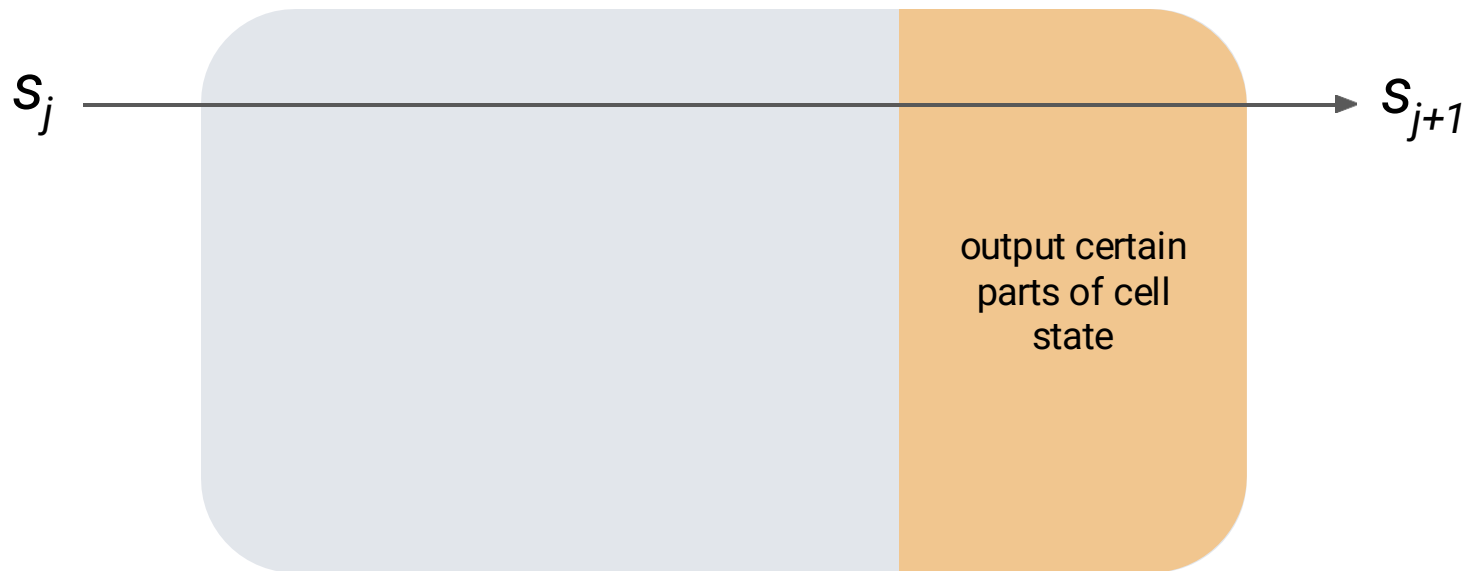
solution #3: more on **LSTMs**



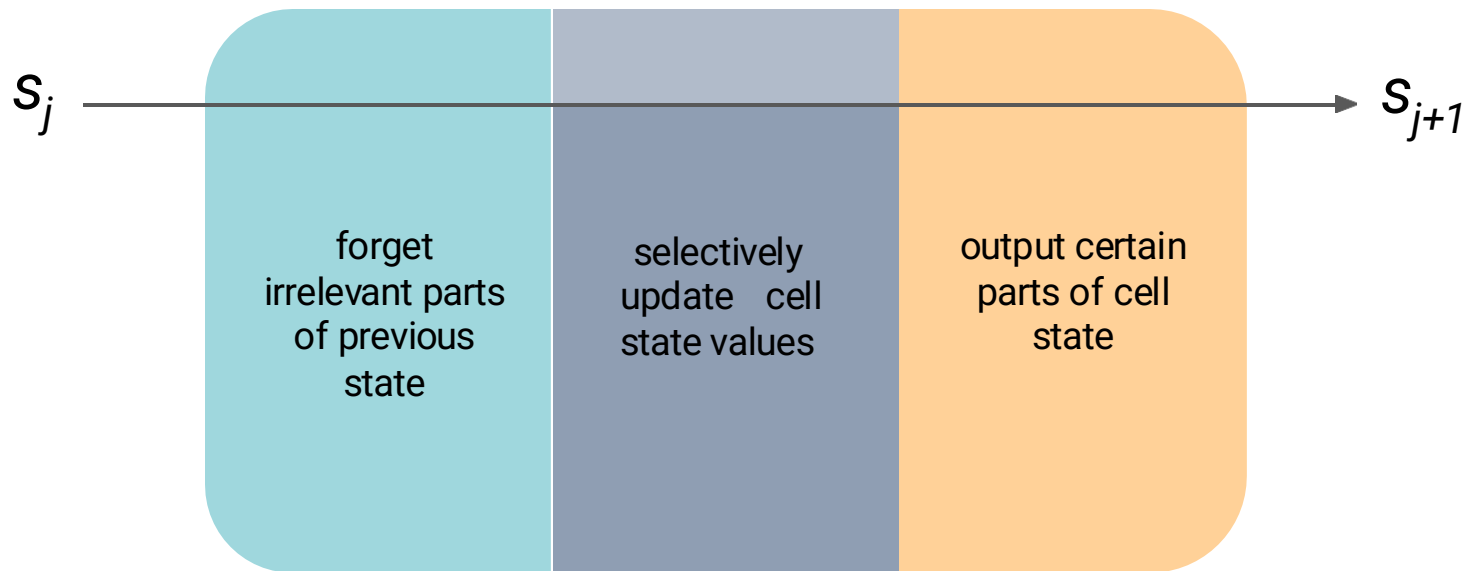
solution #3: more on **LSTMs**



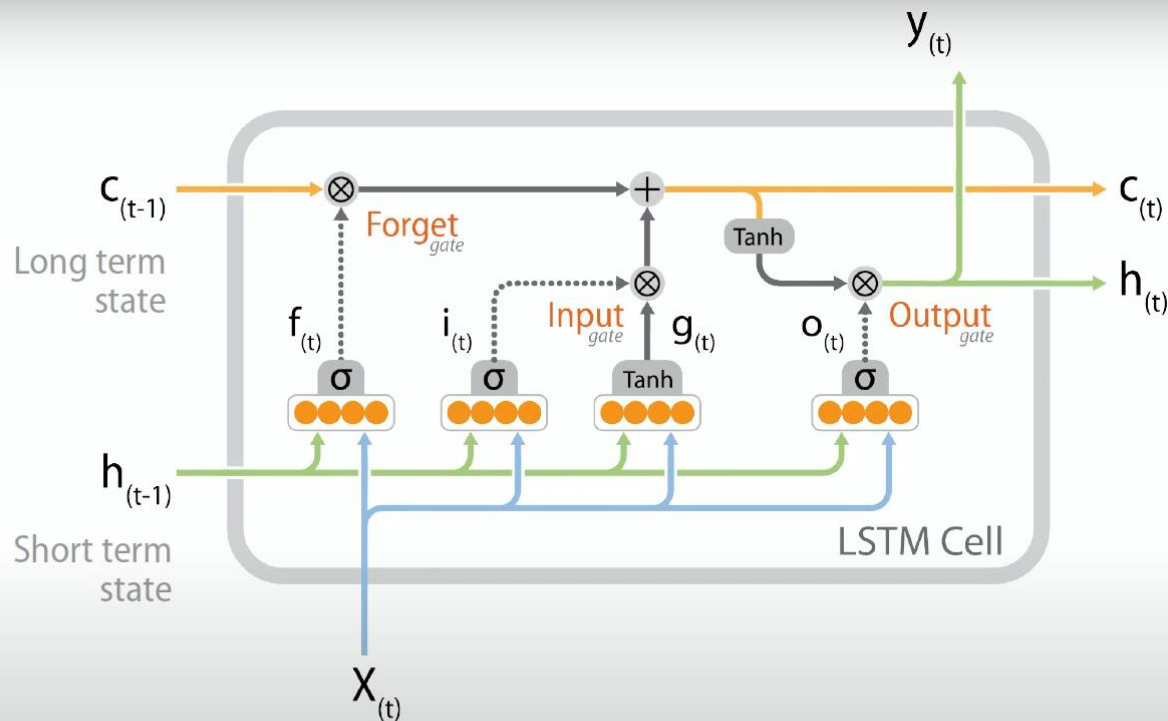
solution #3: more on **LSTMs**



solution #3: more on **LSTMs**



solution #3: LSTMs in details



$$\begin{aligned}f_{(t)} &= \sigma(W_{xf}^T X_{(t)} + W_{hf}^T h_{(t-1)} + b_f) \\i_{(t)} &= \sigma(W_{xi}^T X_{(t)} + W_{hi}^T h_{(t-1)} + b_i) \\g_{(t)} &= \tanh(W_{xg}^T X_{(t)} + W_{hg}^T h_{(t-1)} + b_g) \\o_{(t)} &= \sigma(W_{xo}^T X_{(t)} + W_{ho}^T h_{(t-1)} + b_o) \\c_{(t)} &= f_{(t)} \otimes c_{(t-1)} + i_{(t)} \otimes g_{(t)} \\y_{(t)} &= h_{(t)} = o_{(t)} \otimes \tanh(c_{(t)})\end{aligned}$$

with :

$X_{(t)} \in \mathbb{R}^d$	input vector
$f_{(t)} \in \mathbb{R}^h$	forget gate's activation vector
$i_{(t)} \in \mathbb{R}^h$	input gate's activation vector
$o_{(t)} \in \mathbb{R}^h$	output gate's activation vector
$g_{(t)} \in \mathbb{R}^h$	current entry vector
$h_{(t)}, y_{(t)} \in \mathbb{R}^h$	hidden state or output vector
$c_{(t)} \in \mathbb{R}^h$	cell state vector
$\otimes$	Hadamard product
σ	sigmoid function
W_k	weights matrix
b_k	bias vector

Long short-term memory (LSTM)¹

Gated recurrent unit (GRU)²

why do LSTMs help?

1. forget gate allows information to pass through unchanged

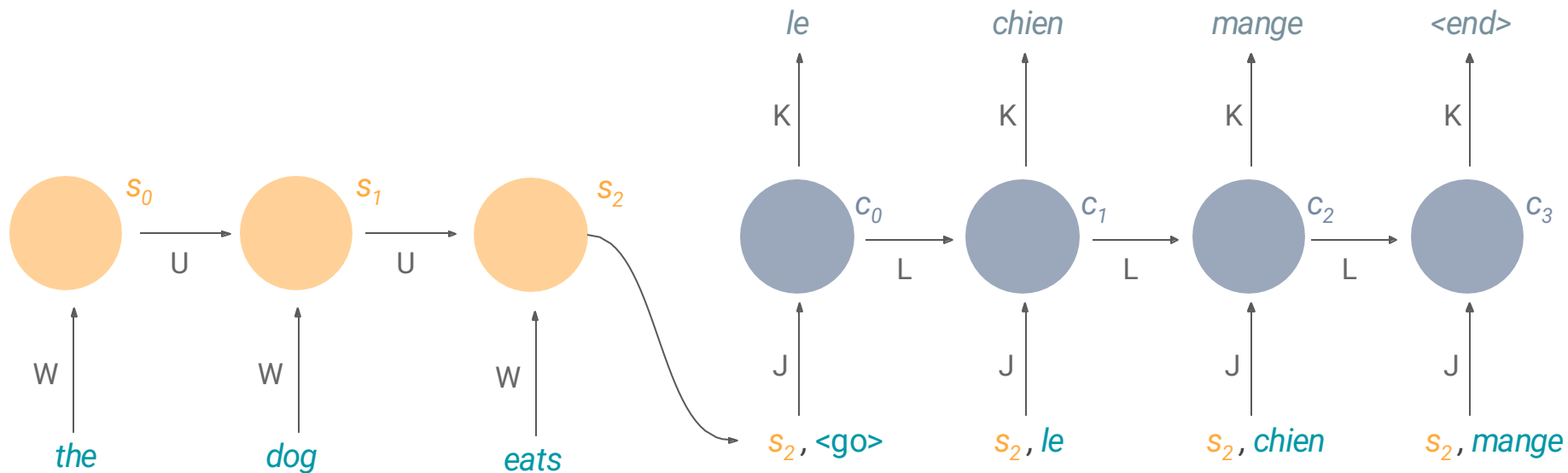
→ when taking the derivative, $\mathbf{f}'$ is 1 for what we want to keep!

2. s_j depends on s_{j-1} through addition!

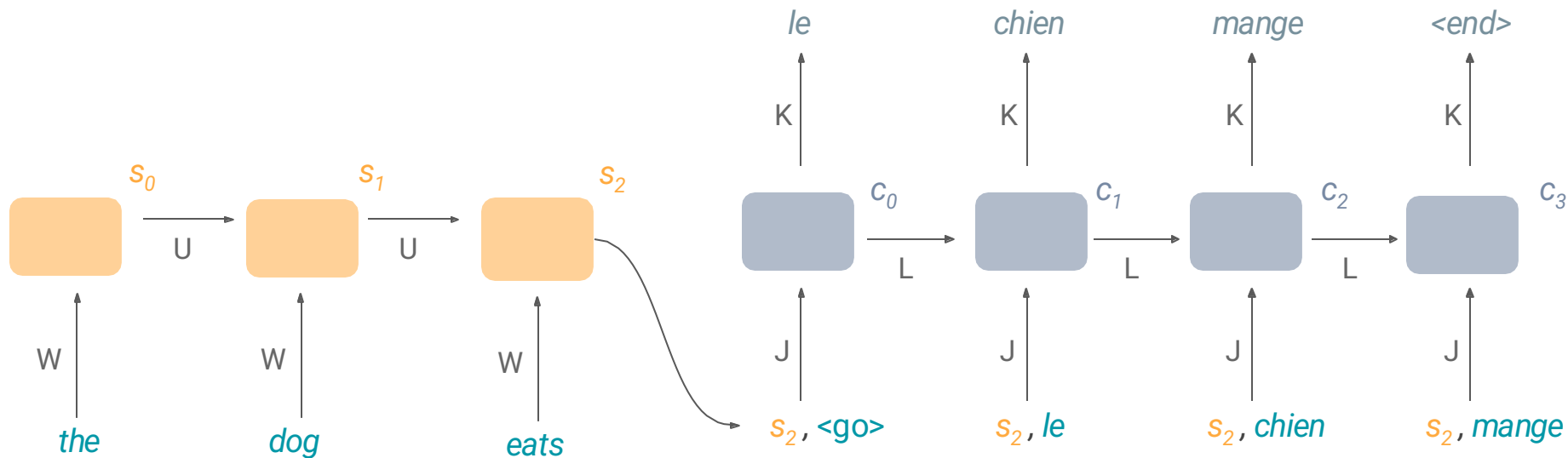
→ when taking the derivative, not lots of small $\mathbf{W}$ terms!

in practice: **machine translation.**

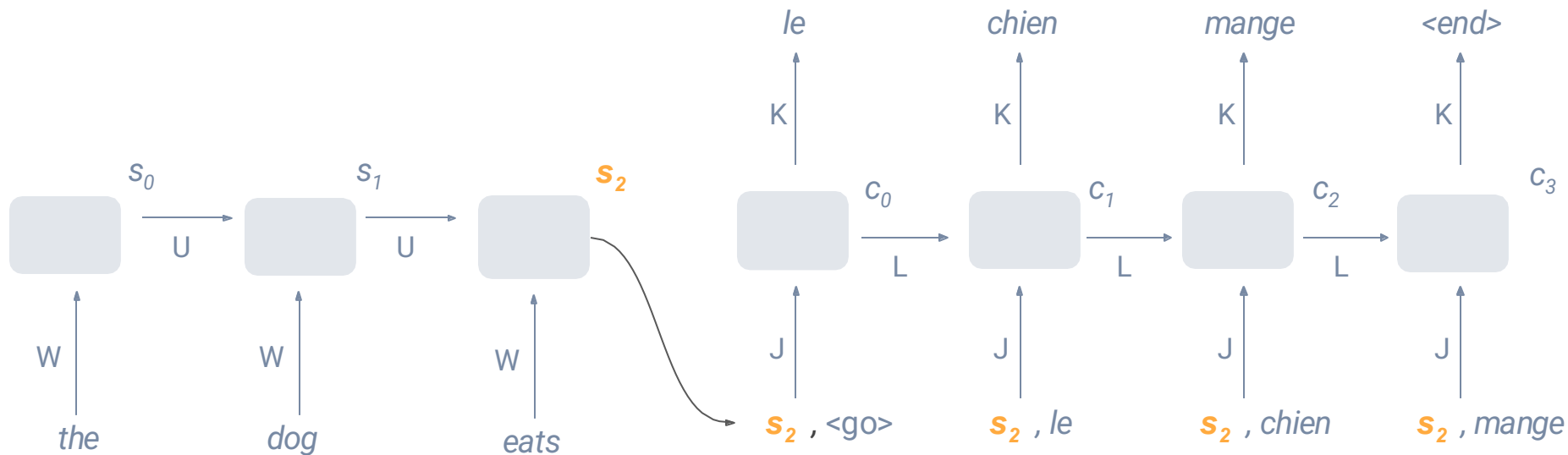
basic encoder-decoder model:



add LSTM cells:

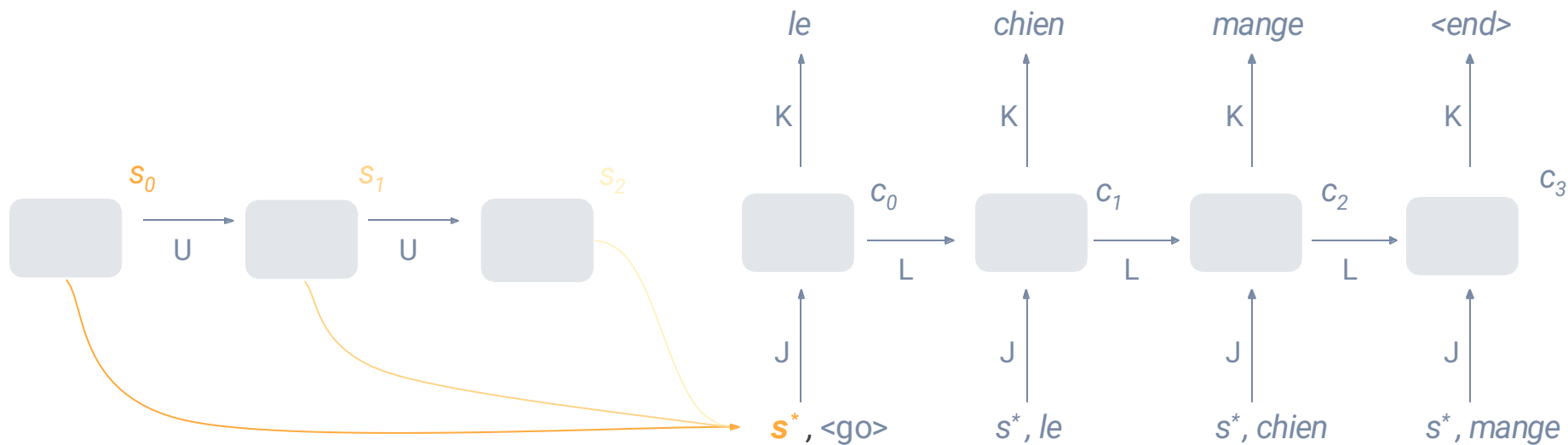


problem: a fixed-length encoding is limiting

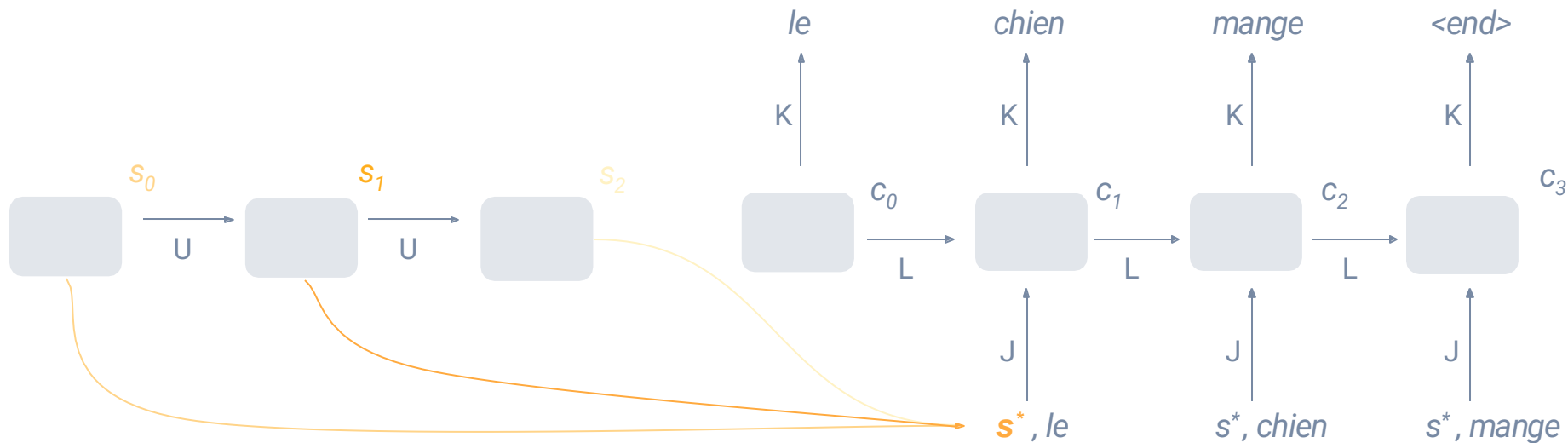


all the decoder knows about the input sentence is in one fixed length vector, s_2

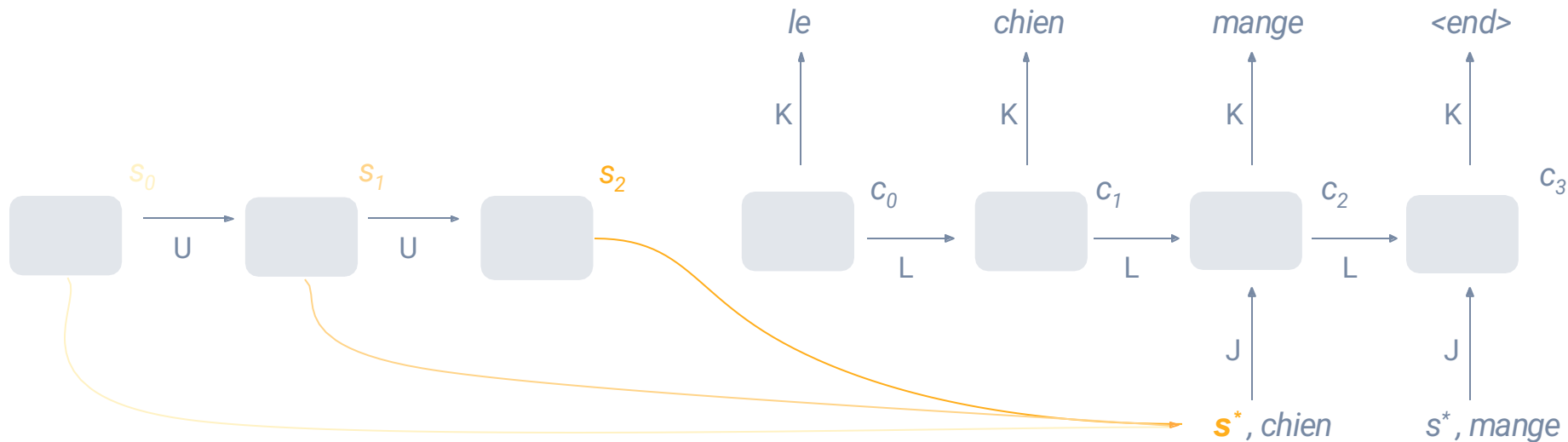
solution: attend over all encoder states



solution: attend over all encoder states



solution: attend over all encoder states



now we can model **sequences**!

- why recurrent neural networks?
- building models for language, classification, and machine translation
- training them with backpropagation through time
- solving the vanishing gradient problem with activation functions, initialization, and gated cells (like LSTMs)
- using attention mechanisms

and there's lots more to do!

- extending our models to timeseries + waveforms
- complex language models to generate long text or books
- language models to generate code
- controlling cars + robots
- predicting stock market trends
- summarizing books + articles
- handwriting generation
- multilingual translation models
- ... many more!